

INFORME DE PROYECTO

To Do List

App móvil con React Native + Expo — Cámara, GPS, autenticación local y sincronización con API externa

Autor: Ignacio Billeke

Plataforma objetivo: Android

Stack: React Native · Expo SDK 54 · TypeScript

Android package: com.ignac.todolist

Gestor de paquetes: pnpm

1. Resumen del proyecto

To Do List es una aplicación móvil desarrollada en **React Native + Expo (SDK 54)** con **TypeScript**, orientada a la plataforma **Android**. Permite a cada usuario registrar y visualizar sus tareas pendientes, adjuntando de forma opcional una foto (tomada con la cámara del dispositivo) y la ubicación GPS donde se creó la tarea.

La app no depende de un backend propio: la autenticación es local (usuarios y contraseñas hasheadas guardados en el dispositivo) y la persistencia de datos se realiza con `AsyncStorage` + `expo-file-system`. Adicionalmente, integra la API pública **JSONPlaceholder** para importar tareas de ejemplo y demostrar sincronización saliente vía `POST`.

Objetivos cumplidos

- Interacción con periféricos: cámara y GPS
- Autenticación de usuarios (registro / login local)
- Integración con servicio web externo (JSONPlaceholder)
- Pruebas automatizadas con Jest (cámara y ubicación mockeadas)

Principios de diseño

- Permisos solicitados de forma *lazy* (nunca al abrir la app)
- Ningún permiso rechazado bloquea el flujo de guardado
- Almacenamiento local como fuente de verdad
- Identidad visual moderna: violeta + coral sobre fondo crema

2. Stack tecnológico

Área	Decisión
Frontend	React Native + Expo Go, TypeScript
Expo SDK	54 (fijo)
Plataforma objetivo	Android
Backend	Ninguno — Node.js solo como tooling de desarrollo (Expo CLI, pnpm)
Persistencia	Local — <code>AsyncStorage</code> + <code>expo-file-system</code>
API externa	JSONPlaceholder (importación + sincronización vía POST)
Navegación	React Navigation (Stack Navigator)
Testing	Jest + jest-expo
Gestor de paquetes	pnpm

Dependencias principales

- expo-camera
- expo-location
- expo-crypto
- expo-file-system
- expo-image-manipulator
- @react-native-async-storage/async-storage
- @react-navigation/native
- @react-navigation/native-stack
- @expo/vector-icons
- jest + jest-expo

3. Modelo de datos

```
type User = {
  id: string;           // uuid generado localmente
  username: string;     // único, case-insensitive
  passwordHash: string; // SHA-256 (expo-crypto), nunca texto plano
  createdAt: string;    // ISO date
};

type Task = {
  id: string;           // uuid generado localmente
  userId: string;       // dueño de la tarea (User.id), obligatorio
  title: string;        // obligatorio
  description?: string; // opcional
  completed: boolean;
  createdAt: string;    // ISO date
  photoUri?: string;    // path local del archivo (filesystem)
  location?: { latitude: number; longitude: number };
  source: "local" | "jsonplaceholder";
  syncedAt?: string;    // ISO date de última sincronización exitosa
};
```

4. Pantallas y navegación

Navegación implementada con **React Navigation (Stack Navigator)**. Dos stacks montados de forma condicional según haya sesión activa (`currentUserId` en `AsyncStorage`):

```
AuthStack (sin sesión)
├─ LoginScreen    - usuario, contraseña, botón "Ingresar", link a registro
└─ RegisterScreen - usuario, contraseña, confirmar contraseña

MainStack (con sesión)
├─ TaskListScreen - lista de tareas (FlatList), FAB "+", Importar/Sincronizar, cerrar sesión
└─ TaskFormScreen - crear / editar / ver tarea: título, descripción, foto, ubicación, completada
```

5. Diseño visual

Dirección: **moderno y amigable**, con bordes redondeados, espaciado generoso y sombras suaves en las tarjetas.

Uso	Color
Primario (acciones, header, FAB)	 #7C3AED — violeta
Acento secundario	 #FF6B6B — coral
Fondo	 #FFFBF5 — crema cálido
Éxito	 #22C55E
Error / eliminar	 #EF4444

6. Funcionalidades implementadas

6.1 Autenticación local

- Registro con usuario único (case-insensitive) y contraseña hasheada con SHA-256 (`expo-crypto`).
- Login compara el hash guardado; en error nunca revela cuál dato falló ("Usuario o contraseña incorrectos").
- Sesión persistida en `AsyncStorage`, se mantiene entre reinicios hasta cerrar sesión explícitamente.
- Multi-usuario en el mismo dispositivo; cada tarea queda asociada a su dueño (`Task.userId`).

6.2 Cámara y ubicación (periféricos)

- Permisos solicitados de forma *lazy*: recién al tocar "Agregar foto" / "Usar ubicación actual".
- Foto: capturada con `expo-camera`, comprimida (`expo-image-manipulator`, máx. 1080px, calidad 70%) y guardada en el filesystem del dispositivo (`expo-file-system`); solo se persiste el path en `AsyncStorage`.
- Ubicación: obtenida con `expo-location`; una vez agregada, se muestra confirmación ("Ubicación agregada ✓") en vez de coordenadas crudas.
- Rechazo de permiso, en el momento o permanente: cancela solo esa acción puntual, la tarea se guarda igual sin ese dato.

6.3 Integración con JSONPlaceholder

- Importación inicial: `GET jsonplaceholder.typicode.com/todos`, tareas marcadas con `source: "jsonplaceholder"`, sin duplicar en reimportaciones.

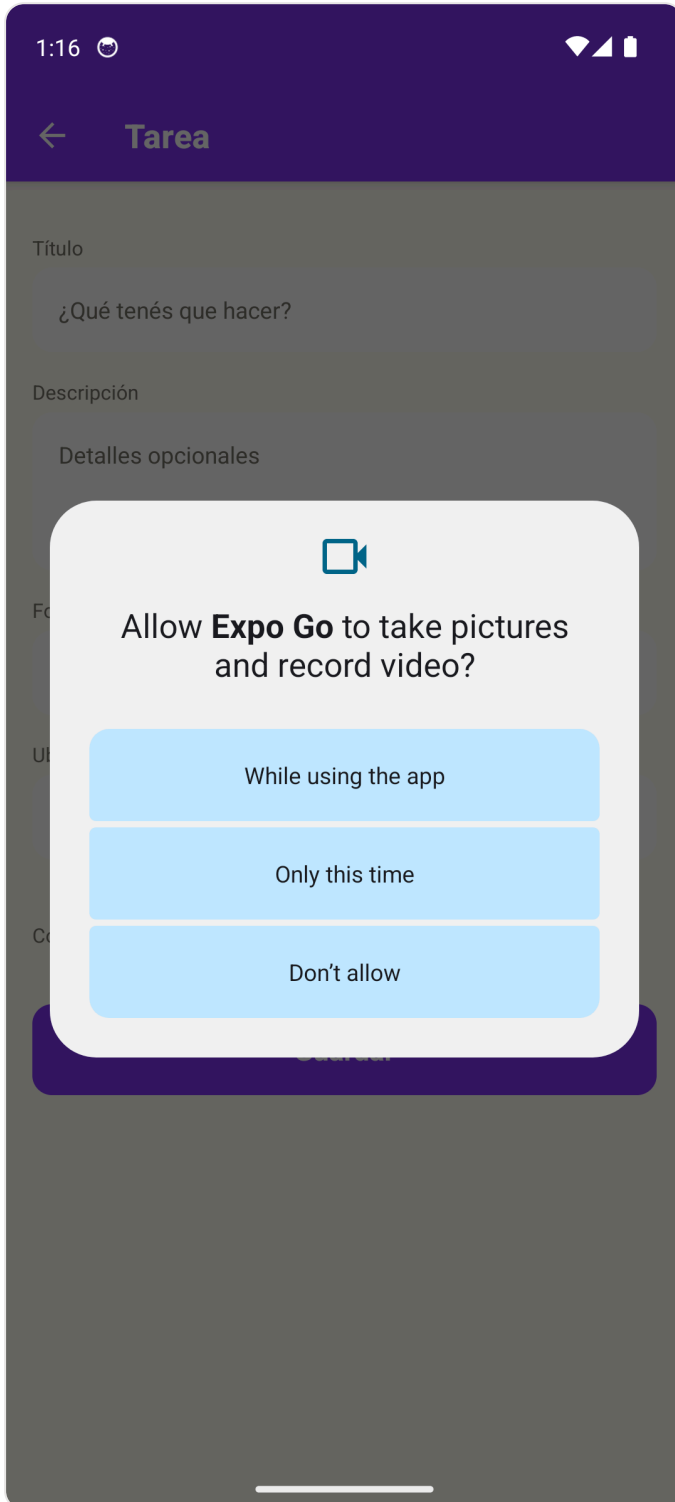
- Sincronización saliente: botón "Sincronizar" hace `POST /todos` por cada tarea local pendiente; al recibir `201` se marca `syncedAt` .
- JSONPlaceholder no persiste escrituras server-side — la fuente de verdad real es siempre `AsyncStorage` en el dispositivo.

7. Evidencia de pruebas: permisos de cámara y ubicación

Pruebas realizadas sobre emulador Android (Pixel 7, Android 14, API 34, imagen [google_apis_playstore](#)), app corriendo vía Expo Go (SDK 54). Flujo probado: *Mis tareas* → FAB "+" → *TaskFormScreen* → "Agregar foto" / "Usar ubicación actual".

7.1 Permiso de cámara

1. Solicitud lazy — diálogo nativo aparece recién al tocar "Agregar foto"



2. Permiso otorgado — se abre la vista de cámara (CameraView)



3. Foto capturada, comprimida y guardada — preview visible en el formulario

1:17

←

Tarea

Título

¿Qué tenés que hacer?

Descripción

Detalles opcionales

Foto

Reemplazar foto

Ubicación

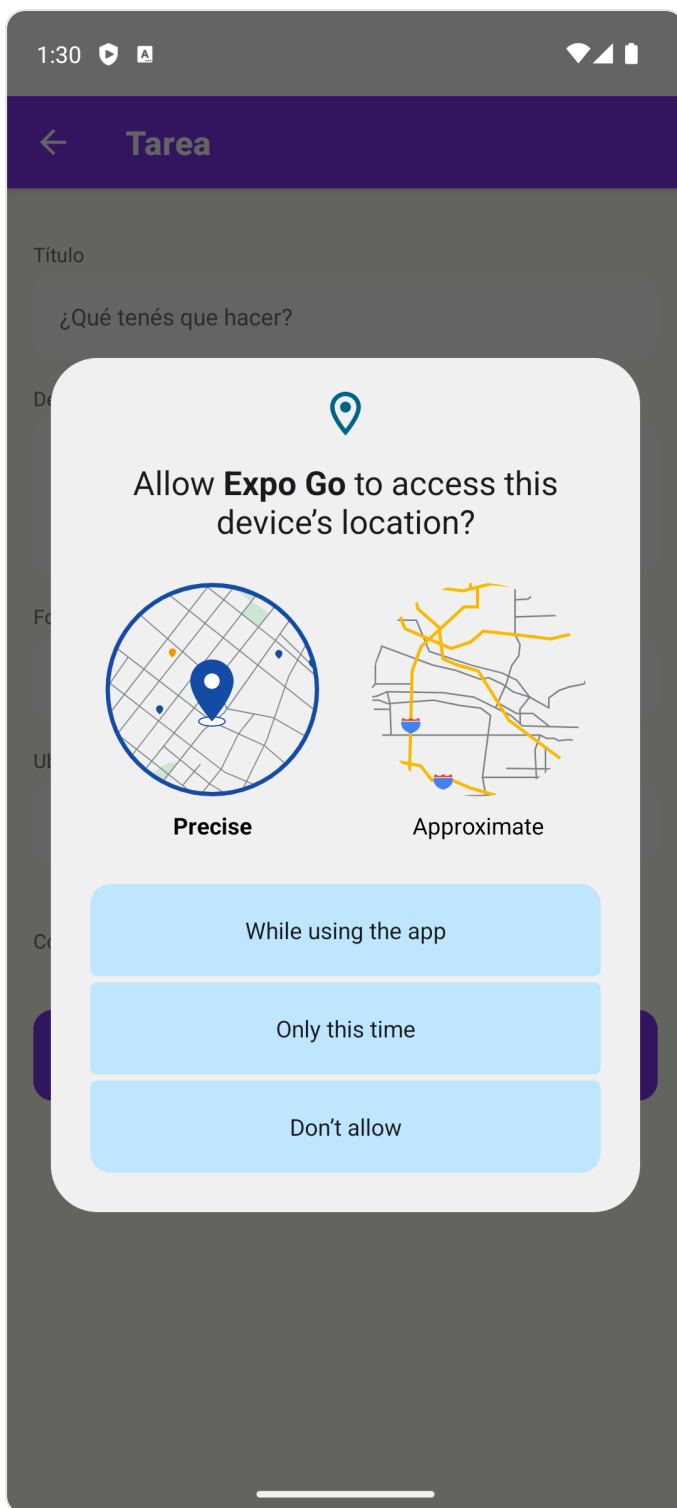
Usar ubicación actual

Completada

Guardar

7.2 Permiso de ubicación

4. Solicitud lazy — diálogo nativo de ubicación (Precise/Approximate)



5. Permiso otorgado, confirmado por el sistema operativo — app en estado de carga

1:30

←

Tarea

Título

¿Qué tenés que hacer?

Descripción

Detalles opcionales

Foto

📷

Agregar foto

Ubicación

C

Completada

Guardar

Nota: la resolución final de la coordenada GPS quedó bloqueada por una limitación conocida del Fused Location Provider del emulador Android (no procesa correctamente fixes inyectados sin dispositivo físico o Extended Controls). El permiso en sí — objeto de esta prueba — quedó otorgado y confirmado por el sistema operativo; el código de la app no arrojó errores durante la prueba.

7.3 Resumen de resultados

Permiso	Solicitud lazy	Diálogo nativo	Permiso otorgado	Acción posterior
Cámara	✓	✓	✓	✓ Foto capturada y guardada

Ubicación	✓	✓	✓ (confirmado por SO)	⚠ Coordinada bloqueada por el emulador (no por la app)
-----------	---	---	-----------------------	--

8. Pruebas automatizadas

Framework: **Jest** con preset **jest-expo**, mockeando `expo-camera` y `expo-location`.

- `TaskFormScreen-test.tsx` — captura de foto (compresión/guardado, `photoUri` seteado) y obtención de ubicación (coordenadas mockeadas, `location` seteado).
- `authStorage-test.ts` — alta de usuario, rechazo de usuario duplicado, login con credenciales inválidas.
- `jsonPlaceholder-test.ts` — integración con la API externa (importación de tareas).

Ejecución: `cd app && pnpm test`.

9. Estructura del proyecto

```
Examen2AppMovil/
├── docs/                documentación (brief, diseño, plan, pruebas)
└── app/                código fuente Expo/React Native
    ├── App.tsx          entry point, providers, stack raíz
    └── src/
        ├── api/         cliente JSONPlaceholder
        ├── auth/         AuthContext (sesión)
        ├── components/   AuthHero, ScreenHeader, TaskCard
        ├── navigation/   tipos de navegación
        ├── screens/      Login, Register, TaskList, TaskForm
        ├── storage/      authStorage, taskStorage (AsyncStorage)
        ├── theme/        colores y estilos
        └── types/         Task, User
```

10. Roadmap (fuera de alcance actual)

- **Firebase Firestore:** reemplaza a JSONPlaceholder como backend remoto real, persiste escrituras y habilita sincronización.
- **Autenticación multi-dispositivo:** migrar de auth local a Firebase Authentication, sincronizando sesión y usuarios entre dispositivos.
- **Sincronización en tiempo real** entre dispositivos, una vez migrado a Firestore.

11. Conclusión

El proyecto cumple los cuatro ejes solicitados: interacción con periféricos de hardware (cámara y GPS), autenticación de usuarios local con contraseñas hasheadas, integración con un servicio web externo (JSONPlaceholder, con importación y sincronización real vía POST) y pruebas automatizadas con Jest sobre los componentes de hardware. La app funciona de forma completamente local — sin backend propio — usando `AsyncStorage` como fuente de verdad, dejando planificada la migración a Firebase Firestore como evolución futura fuera del alcance actual.